

Docker

1. Types of application -- standalone , Web based , Client server application
2. Types of application architecture -- monolithic , microservices , 3 tier architecture
3. Introduction to containers
 - what is containers?
 - containers v/s virtual machine
4. Introduction to docker
 - what is docker?
 - history of docker
 - features of docker
 - alternative tools
 - docker architecture
 - basic commands in docker
5. Docker files -- docker file examples
6. Docker volumes
 - why we need docker volumes?
 - types of docker volumes
 - basic commands in docker volume
7. Docker network
 - why we need docker network?
 - types of docker network
 - basic commands in docker network
8. Multistage dockerfile
 - why we need multistage dockerfile?
 - multistage dockerfile implementation
9. Docker compose
 - why we need multistage docker compose?
 - introduction to YAML file
 - docker compose YAML file examples
10. Integration to docker with jenkins
11. Introduction to ECS & ECR

Docker

Types of application

1. Stand alone application

It doesn't require internet connection to use the application and can run independently on local system/computer.

Ex : Paint, caculator, notepad, offline games, ms word etc

2. Web based application

It requires the internet connection to access via browsers and it runs on web server.

Ex : Gmail, netflix, whatsapp, youtube, facebook, amazon, linkedln, instagram etc

3. Client-server application

It has two parts -- Client : A front end software that user will interact with.

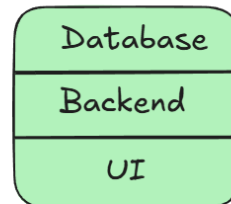
Server : A back end system that will store the data and handle the requests done by users.

The communication between them is done over network.

Ex : Instagram, whatsapp, amazon, flipkart etc

Types of application architecture

1. Monolithic



All the components such as UI, backend and database is implemented in a single large server.

They are built and also deployed together that means it runs as one program.

Advantages

1. Initially testing and deploying is very easy as it will run as one program
2. Initially developing is also very simple/easy

Drawbacks

1. It is difficult to maintain
2. If there is an issue in any one of the component the entire application fails
3. It is not easy to fix it as it is integrated together.
4. It is difficult to scale as it requires redeployment if changes are done in any of the components.

2. Microservices

A software is divided into small services that are independent.

Each and every services will have their own specific function & runs separately.

Ex : payment page, login, add to cart, notification etc

Note : They communicate with each other through APIs

Advantages

1. It is easy to scale
2. It is easy to make changes and as the services are independent it doesn't affect if any changes are done.
3. Easy to update & maintain.

Disadvantages

1. If it is not handled properly then the communication between the services fails.
2. As it is very complex you need tools such as docker & k8's to manage it.

3. 3-tier architecture

In this it's divided into 3 layers :

1. Presentation layer
2. Logic layer
3. Data layer

1. Presentation layer

- >> It is the front end of an application and users will interact with the application in this layer.
- >> Here when the user interacts the request will be sent to the further layer that is logic layer and then receives the respond.

Ex : HTML, JS, CSS, ReactJS, Bootstrap etc

2. Logic layer

- >> It is the back end of an application and it processes the user request then it interacts with the data layer.
- >> It is also called as business layer as it consists of core part of an application basically calling it as brain of application
- >> It handles decision making.

Ex : Java, Python, C++, C# etc

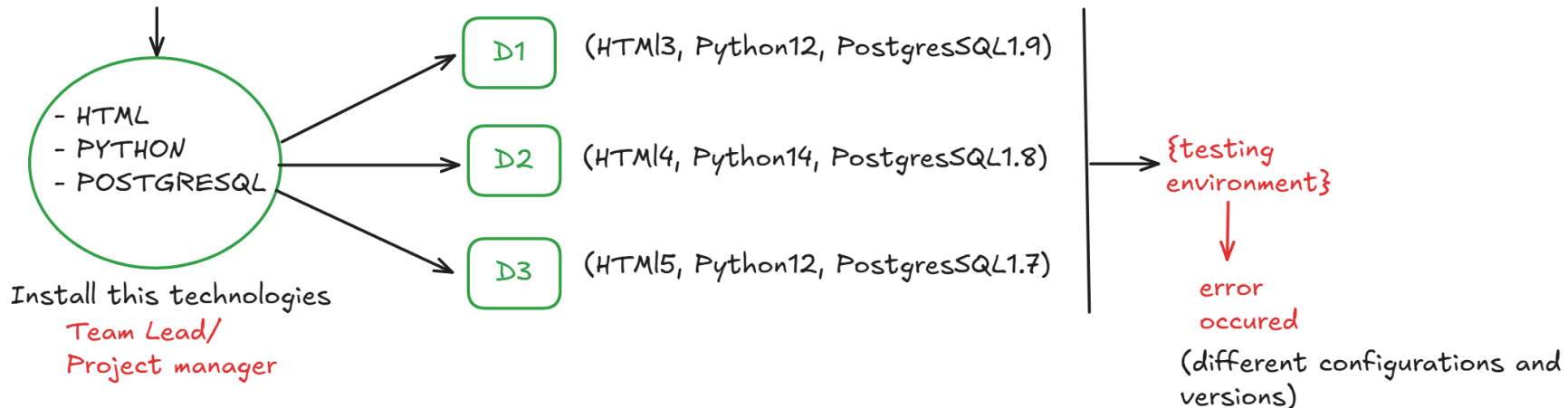
3. Data layer

- >> It contains & manages the data of the software in the database.
- >> It receives the request done by user in order to validate, processes it and sends back the respond to logic layer.

Ex : MySQL, PostgreSQL, MongoDB, OracleDB etc

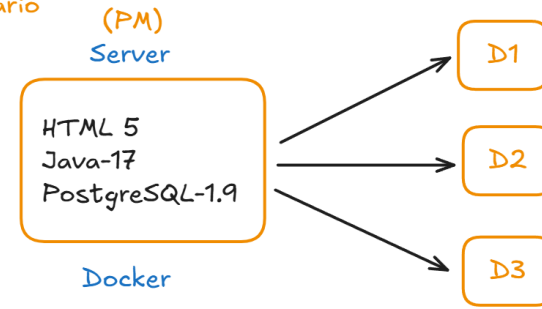
Why do we need containers ?

E-commerce/Banking application

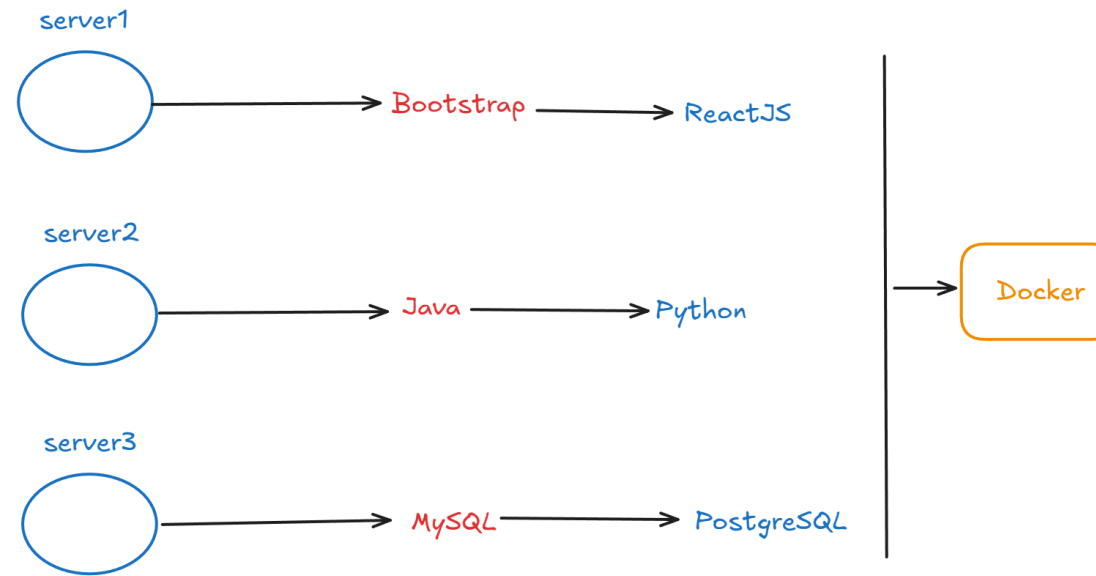


In order to overcome the issue,

1st scenario



2nd scenario



Introduction to container

What is container?

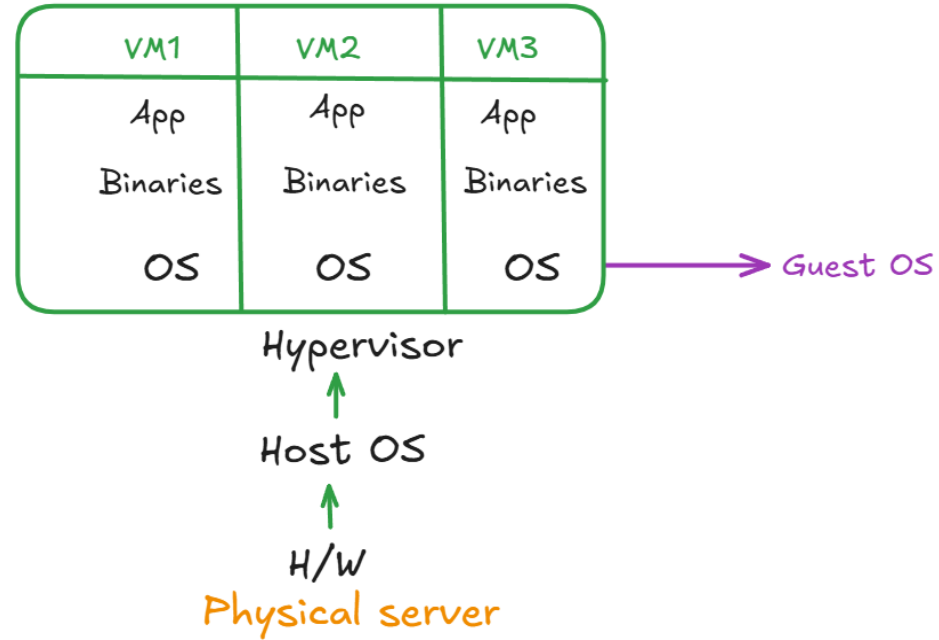
>> It is the executable package which has everything that an application requires.

It requires :

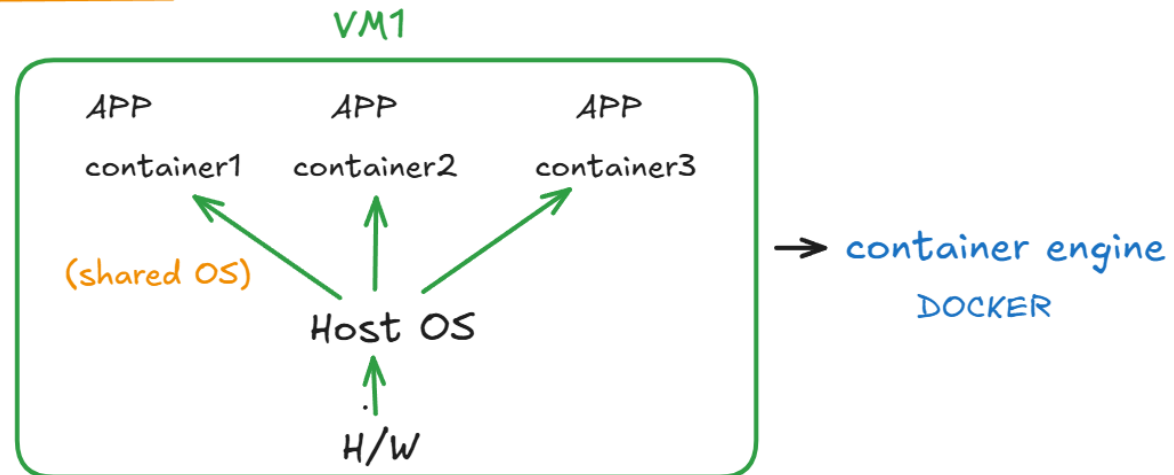
- Dependencies
- Pluggins
- Runtime environment
- Libraries
- Executable packages
- Source code of software

Difference between virtual machine and containers

Virtual machine



Containerization



Introduction to Docker

What is docker?

>> It is free & open source platform where we will run the application in the form of containers.

History of docker

>> 2010 - The person named soloman hykes, he invented dotcloud where it was able to create containers

>> 2013 - Docker was invented as free & open source platform where we can create containers for storing the application.

>> 2014 - Docker Inc launched dockerhub for storing the images for the containers to run application.

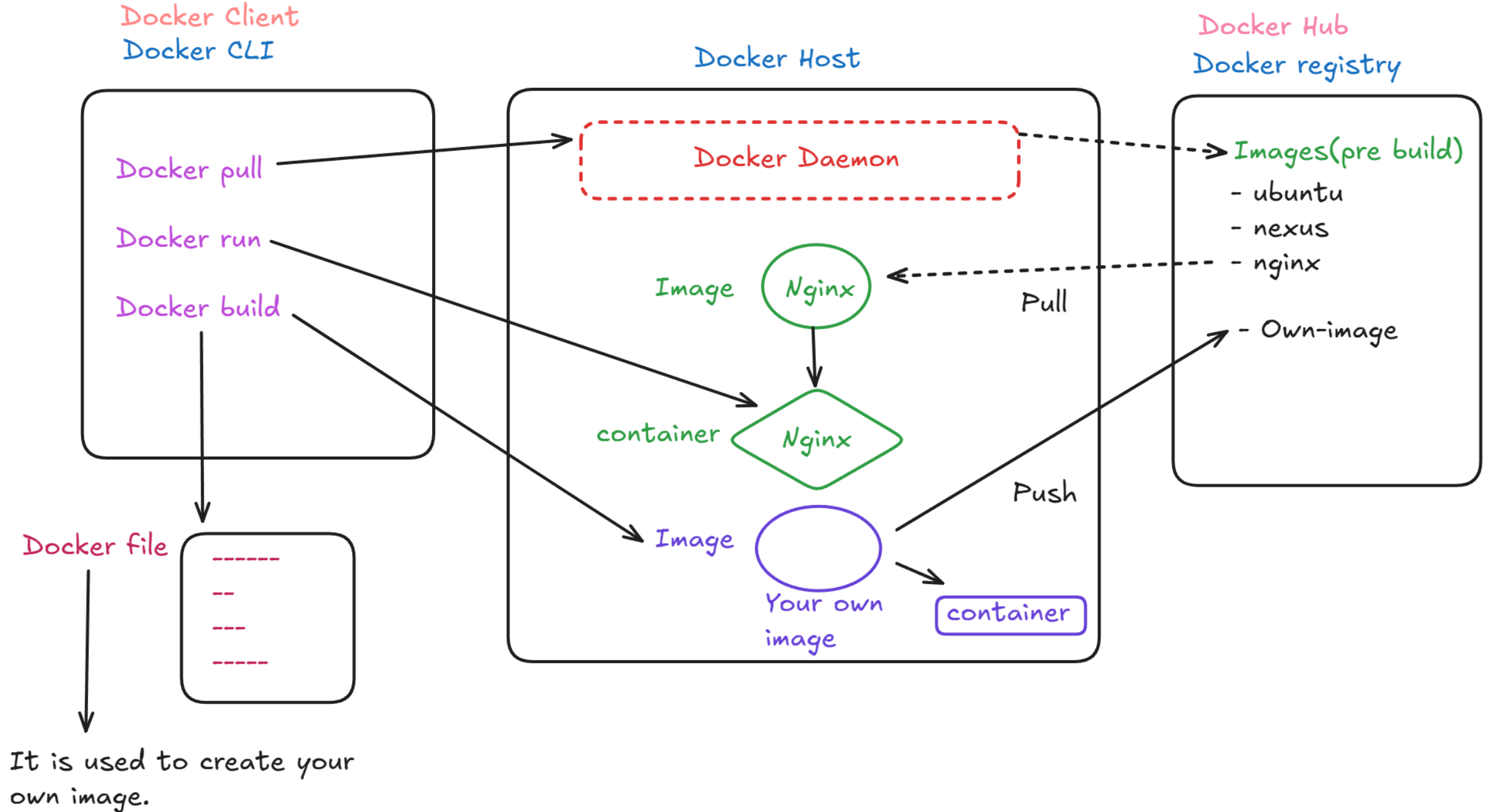
Features of docker

1. Free & open source
2. Supports containerization techniques
3. Has dockerhub to store images
4. Easy to use & manage
5. Can be integrated with jenkins too
6. Lightweight & fast
7. Portable - works same in every system as it has same configuration
8. Scalable - Can run multiple containers for balancing/distributing the load.

Alternative tools

1. Podman
2. LXC/LXD
3. RKT(Rocket)
4. Containerd
5. CRI-O

Architecture of docker



Docker architecture components

1. Docker client (Docker CLI)
 - >> It is the tool that is used to interact with docker by providing docker commands.
 - >> It is remote control that gives instructions to docker to perform the task
 - Ex : docker run , docker build , docker pull etc
2. Docker host
 - >> It is the system/machine (virtual or physical) where we can create our own image, pull the images, create & manage the containers of own image or pre-build images.
 - >> It is the main part of docker architecture where docker daemon runs.
3. Docker daemon
 - >> It is the service running in background that does all the necessary work that is building, running & managing the containers where application is stored.
4. Docker registry
 - >> It is the remote repository that consists of all images (pre-build, own, shared)
 - >> You can pull the images that is required & push your own image as well.
 - Ex : docker hub
5. Images
 - >> It is the blueprint or template that is used to create the containers.
 - >> It includes source code of the application, binaries, dependencies, libraries & information of how to run/use the application.
 - Ex : Nginx, PostgreSQL, Ubuntu, Python, MongoDB, httpd etc

Basic commands of docker

1. docker pull <image_name> - Used to pull image from docker hub.
2. docker images - Used to list all the images that is created or pulled.
3. docker run <image_name> - Used to create the container from the image.
 - docker run -it <image_name> - Used to create the container in interactive mode
 - docker run -d <image_name> - Used to create the container in detached mode.
4. docker ps - Used to list all running containers
5. docker ps -a - Used to list all the running as well as stopped containers.
6. docker start <container_name>/<container_ID> - Used to start the stopped containers.
7. docker stop <container_name>/<container_ID> - Used to stop the running containers.
8. docker rm <container_name>/<container_ID> - Used to delete the stopped container.
9. docker rmi <image_name> - Used to delete the image.

Short term containers

- >> These containers will run only for short period of time & it stops automatically as soon as the task is completed.
Ex : OS --> ubuntu, debian , programming language --> python etc

Long term containers

- >> These containers will run for long time as it is designed to run continuously providing the applications/software to run.
- >> It will stop only when we manually stop or terminate the container.
Ex : nginx, mysql etc

Commands

1. To stop all the running containers at once
--> `docker stop $(docker ps -q)`
--> `docker stop containerid1 containerid2 containerid3`
2. To delete all the stopped containers at once
--> `docker rm containerid1 containerid2 containerid3 containerid4`
--> `docker rm $(docker ps -aq)`
3. To delete all the images at once
--> `docker rmi $(docker images -q)`
4. To delete the images forcefully
--> `docker rmi -f $(docker images -q)`
5. To create container without pulling image
--> `docker run -d nginx`
6. To give your own container name while creating it
--> `docker run --name <own_container_name> <image_name>`
7. To delete the running container forcefully
--> `docker rm -f <container_name>/<containerId>`
8. To restart the container
--> `docker restart <containerId/Name>`
9. To check the log of container
--> `docker logs <containerId/Name>`
10. To inspect the container
--> `docker inspect <containerId/Name>`

Docker files

- >> It is the text file which is used to create/build our own image.
- >> The text file contains of set of instructions(steps) which helps 4 to create an image.

Structure of docker file

Instructions

1. FROM <base_image_name>

- >> It is used to specify the base image on which the new image has to be created.
- >> Basically it is the base image of the docker file.

2. WORKDIR <directory_name>

- >> It is used to create the directory inside the container.

3. COPY <source_path> <destination_path>

- >> It is used to copy the files from source path to the destination path.
- >> It can also copy the folders from host(local machine) to the container.

4. RUN <commands>

- >> It is used to run the commands during the image creation/build process.
- >> It is basically used for updating the executable packages, installing applications etc.
- >> Everytime if you use RUN command it will create the new layer of image.

5. CMD ["commands", "args"]

- >> It is used to run the commands as soon as the container is started.
- >> Once the container is started the commands will get executed.
- >> CMD can be overridden

6. ENTRYPOINT ["commands", "args"]

- >> It is used to run the commands or execute it during container creation process.
- >> ENTRYPOINT can't be overridden
- >> ENTRYPOINT is fixed.

Different between CMD & ENTRYPOINT

CMD	ENTRYPOINT
<u>Docker file</u> FROM nginx CMD ["echo","hello"] O/P --> hello docker run <myownimage> echo "bye" O/P --> bye	<u>Docker file</u> FROM nginx ENTRYPOINT ["echo","hello"] O/P --> hello docker run <myownimage> echo "bye" O/P --> bye

7. EXPOSE <port_number>

- >> It is used to expose the port number for the containers in order to communicate.
- >> The application inside the container will listen to the specified port number.

Note : The name of dockerfile should be by default "Dockerfile".

Q] How to create the image from dockerfile?

>> docker build -t <image_name>

Q] How to use/demonstrate expose in docker file?

>> Dockerfile --> FROM nginx
COPY ./usr/share/nginx/html (default nginx path)
EXPOSE 80

To publish the port number while we are running the container we should go for the concept called as port mapping

command : docker run -p <host_portnumber>:<container_portnumber> <image_name>

Q] How to push the own image into dockerhub?

>> 1. login to dockerhub/it'll give access to the dockerhub
--> docker login

2. To tag the image

--> docker tag <image_name> <dockerhub_ID>/<image_name>:<version>

3. To push the image into dockerhub

--> docker push <dockerhub_ID>/<image_name>:<version>

Docker volumes

- >> Docker volume is used to isolate the data from the container.
- >> It is also used to persist the data that is been created/generated & used by the container.
- >> These volume will be attached to the containers.
- >> It is used to store the data outside the containers writable layer so that the data won't be deleted when the container is deleted.

To save the data permanently the docker provides us with storage options :

1. Volume
2. Bind mounts

1. Volume

- >> It is used to store the data that is completely managed by the docker
- >> It is stored inside the storage area of docker.
- >> If you want the created data inside the container to be saved permanently then we have to go for volumes.

Benefits of volume

1. Can expect persistent data
2. Completely managed by docker
3. It can be attached to multiple containers
4. Easy to backup
5. Stores it in the directory of docker in the local system.

Basic commands of docker volumes

1. To create the volume
 >> docker volume create <volume_name>
2. To list the created volumes
 >> docker volume ls
3. To inspect the volume
 >> docker volume inspect <volume_name>
4. To delete the volume
 >> docker volume rm <volume_name>
5. To attach the volume to the container
 >> docker run -it -v <volume_name>:/<container_path> <image_name>
6. To get in the container to execute the commands
 >> docker exec -it <container_name> /bin/bash
7. To attach the multiple volumes to the container
 >> docker run -it -v <volume_name>:/<container_path> -v <volume_name>:/<container_path> --name <container_name> <image_name>

Note : Delete the container before deleting the volume.

Bind mounts

- >> Bind mounts are used to link the files & folders of host that is local system to the path of container.
- >> The files & folder that are present in the local system will be reflected inside the container.
- >> If you make any changes in any of the path it will reflect in the container & localhost.
- >> It gives complete and direct access to the local host filesystem.
- >> It is also useful & flexible when it comes to development.

Command

docker run -it -v <local_system_path>:/<container_path> <image_name>

Docker network

>> It is used to create our own network where in we can create and connect the containers to run our application within our own network.

Types of docker network :

1. Bridge network
2. Host network
3. None network

1. Bridge network

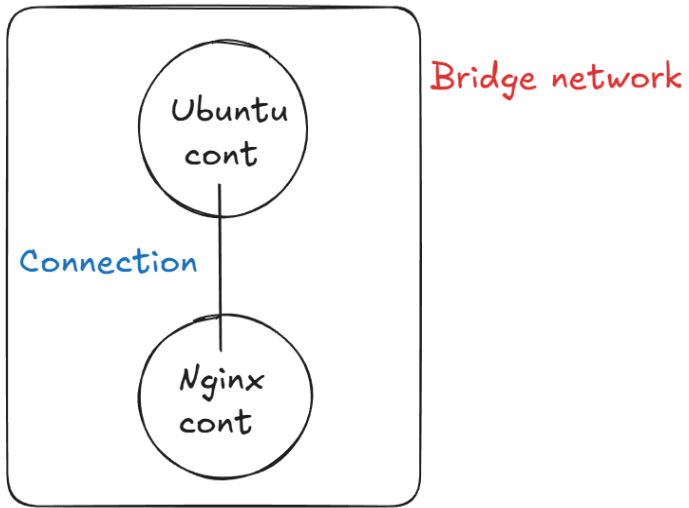
- >> It is the private network where containers are present by default.
- >> It is the default network that connects multiple containers with each other, host(local system) & other networks.
- >> Here, containers will get its own IP address.
- >> By default the name of the network is bridge network.
- >> Usually bridge network will be present in docker by default

Basic commands in docker network

1. To create our own network
 - >> `docker network create -d <driver_name> <network_name>`
2. To list all the network that is created as well as default network
 - >> `docker network ls`
3. To inspect the network
 - >> `docker network inspect <network_name>`
4. To connect the network to the containers
 - >> `docker network connect <network_name> <container_name>`
5. To disconnect the network from the container
 - >> `docker network disconnect <network_name> <container_name>`
6. To delete the network
 - >> `docker network rm <network_name>`

Note : To convert the short term container into long term container
>> `docker run -itd --name <container_name> <image_name>`

S1 : To connect containers to the network(bridge)



Install ping to check the connection between containers

```
>> apt update
>> apt install iputils-ping -y
```

2. Host network

>> Here, the container will be attached or connected with the host network where in the IP address will not be provided but it'll share the host IP (local system)

Command :

```
docker run --network host <image_name>
```

3. None network

>> Here, the container doesn't have any network connectivity at all
>> As it has no connection for the containers that simply means there will be no communication with other containers as well as host.
>> The IP address won't be provided by docker as well.

Use cases :

1. Running program or code
2. Testing purpose
3. Offline task(update/modification)

Command :

```
docker run --network none <image_name>
```

Multi stage docker files

Restaurant

Biryani

- rice
- water
- spices
- oil
- chicken
- salt
- vegetables

1st stage

- cutting the vegetables
- keeping all spices together
- mixing the ingredients
- marinating the chicken
- preparing the chicken biryani

2nd stage

- decorate plate with side dishes
- serve biryani and have it

↓
final result

Similarly,

To develop an application & convert it into image

Application

- Source code
- plugins
- dependencies
- libraries
- binary files
- packages
- editor
- final result

→ Image

>> can expect the size of image to be more

Build stage

- source code
- packaging
- dependencies
- plugins
-
- ...

Final stage/production stage

- final output of the application

→ Final image size < original image size

Definition :

>> Multi stage docker file is basically used to reduce the size of the image and to achieve it multi stage docker file lets you use multiple FROM instruction through different stages in one docker file

Sample Dockerfile :

#build stage

```
FROM ubuntu AS build # build is the alias name
WORKDIR /sample
COPY . /sample
RUN apt update && apt install openjdk-17-jdk -y && javac Test.java
```

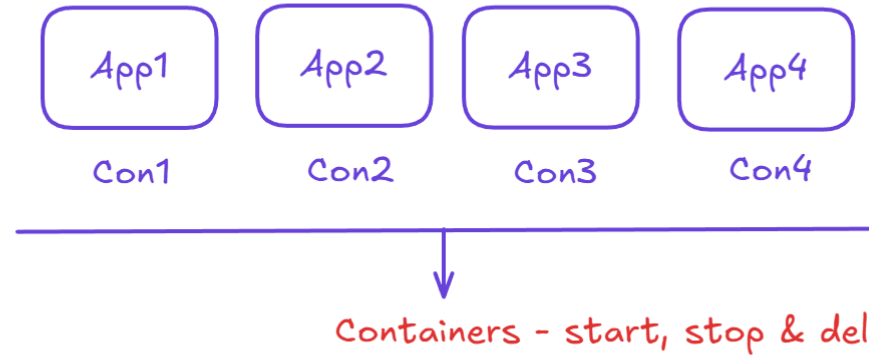
#final stage

```
FROM alpine
WORKDIR /demo
COPY --from=build /sample /demo
CMD [ "java","Test" ]
```

Note : If Dockerfile already exists you cannot create another file with the same name because VS Code does not allow duplicate filenames. So, to create an additional Dockerfile, you must name it followed by a dot and a custom name (example: Dockerfile.<customname>).

Docker compose

>> It is the tool that is used to create, run & manage multiple containers together which contains the application.



Advantages

1. Setup of multiple containers
2. It automates.
3. Configuration is easy with the help of YAML file
4. Provides portability

YAML file

- >> It stands for Yet Another Markup Language -- old name
>> YAML Ain't Markup Language -- new name
>> It is the text file that represents the data serialization format (structured data -- key-value pair)
- create the containers
 - manage the containers
 - a) create & manage volumes
 - b) create & manage networks
 - c) create & manage images
 - d) Assign port number

Sample YAML file syntax

services:

<name_of_service>:

image: <image_name>

volumes: #attaching the volume

- <vol_name:/container_path>

networks: #attaching the network

- <network_name>

ports:

- "<host_portno>:<container_portno>"

→ Container 1

<name_of_service>:

image: <image_name>

volumes: #attaching the volume

- <vol_name:/container_path>

networks: #attaching the network

- <network_name>

ports:

- "<host_portno>:<container_portno>"

→ Container 2

volumes: #create the volume

<name_of_volume>:

networks: #create the network with driver name

<name_of_network>:

driver: <driver_name>

Docker compose commands

1. docker compose build

>> Used to build the custom image defined in the docker compose file.

2. docker compose up

>> Used to start or run all containers mentioned in the compose file.

3. docker compose up -d

>> Used to run containers in detached mode.

4. docker compose stop

>> Used to stop all running containers from the compose file.

5. docker compose start

>> Used to start all stopped containers from the compose file.

6. docker compose down

>> Used to stop and remove containers and networks created by the compose file.

7. docker compose exec <service_name> <command>

>> Used to run a command inside a running container.

8. docker compose ps

>> Used to list containers managed by docker compose.

9. docker compose logs

>> Used to display logs of all services.

10. docker compose logs <service_name>

>> Used to display logs of a specific service.

Steps for integrating Docker with Jenkins

1. Install the docker plugin in jenkins and add your dockerhubID credentials in credentials (Manage jenkins --> Credentials --> System --> Global credentials (unrestricted) --> Add credentials).
 - Select username with password as kind & specify scope, username, password and ID then click on create.
2. Create a new pipeline job & add a clear description in the job configuration page.
3. Write your pipeline script using sample docker project in "Git-clone" stage and add command to build image in "Image-creation" stage.
4. In stage of "Push image in dockerhub" use "withCredentials: Bind credentials to variables" to add credentials.
 - Click on add bindings and select username & password (separated) then specify username variable and password variable and add docker credentials in credentials section then generate the script then add commands to login, tag image and push image in dockerhub.
5. Apply, save and click on build now then check dockerhub for confirmation that the image have been pushed in dockerhub.

Pipeline script

```
pipeline {
  agent any
  stages {
    stage('Git-clone') {
      steps {
        git branch: 'main', url: 'https://github.com/TANVIP-6M/dockerf.git'
      }
    }
    stage('Image-creation') {
      steps {
        bat 'docker build -t a7image .'
      }
    }
    stage('Push image in dockerhub') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub', passwordVariable: 'DOCKER_PASS', usernameVariable: 'DOCKER_USER')]) {
          bat '''echo %DOCKER_PASS% | docker login -u %DOCKER_USER% --password-stdin
              docker tag a7image %DOCKER_USER%/a7image:latest
              docker push %DOCKER_USER%/a7image:latest'''
        }
      }
    }
  }
}
```

ECR - Elastic Container registry

- >> It is the service provided by AWS which is used to store the images in public and private repository.
- >> It is used to store, manage the images securely/safely.

Why do we need ECR ?

1. To keep the image private and safe
2. Supports integration with CI/CD pipeline
3. Possible to integrate with ECS & EKS
4. Highly scalable
5. Supports access control through IAM.

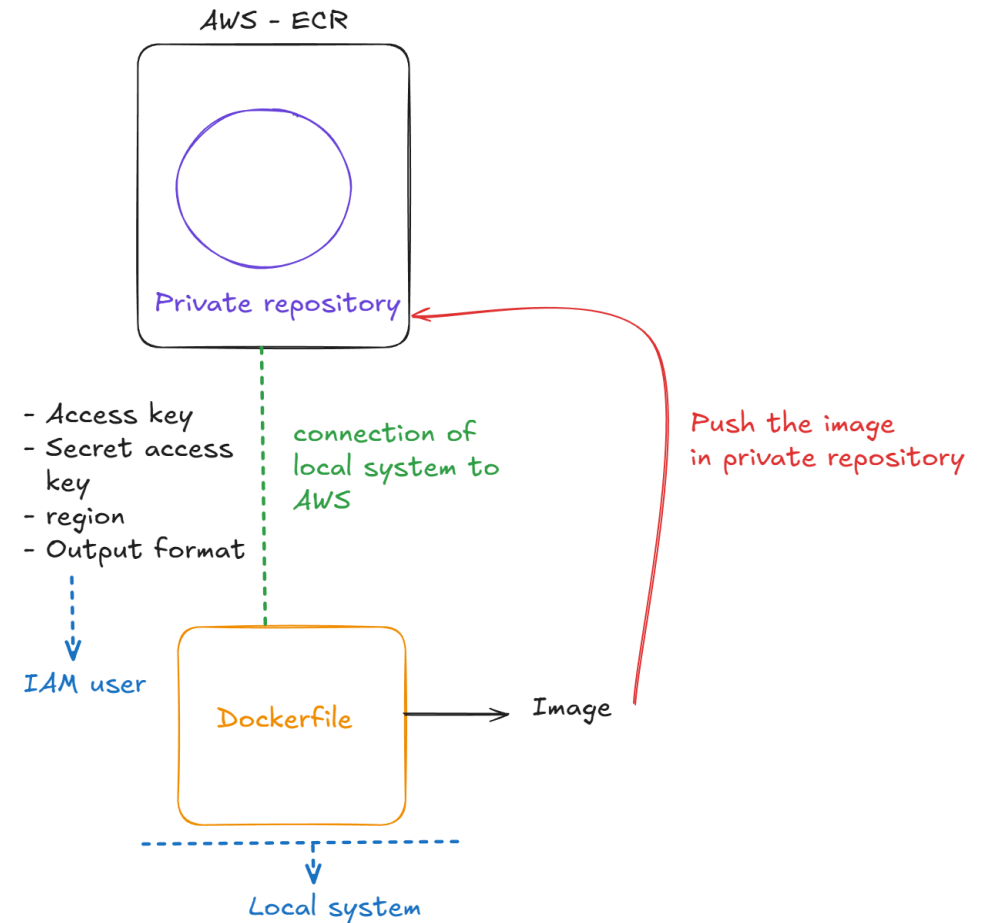
2 types of repositories in ECR :

1. Private repository
2. Public repository

1. Private repository

- >> Private repository is used to store images in it so that other users can't access it apart from the authorised users.
- >> It is secure and is used company projects.

Q] How to push image in private repository after creating it in ECR?



Steps to push image in ECR private repository.

1. Open the IAM service, go to users and create a new user by providing a username.
2. Choose attach policies directly & search for ec2containerregistryfullaccess, select it and continue.
3. Complete the user creation and open the user's security credentials section.
4. Go to access keys, create a new access key, choose the CLI option and proceed.
5. Note the access key id and secret access key displayed after creation.
6. Run the command `aws configure` in the command prompt and enter the access key id, secret access key, preferred region and set the output format to json.
7. Open the ecr service in aws, go to repositories and choose to create a new private repository by giving a name and confirming the creation.
8. Login to ecr, open the created repository and view the push commands.
9. Click on view push commands in the repository.
10. Copy the login command, run it in the command prompt and ensure the login is successful.
11. Make sure you have a dockerfile ready to build into an image.
12. Copy the docker build command to create the image, tag it and push it to the ECR private repository.

Steps to push image in ECR public repository.

1. Configure AWS credentials using your access key id and secret access key.
2. Ensure the selected region is set to N.Virginia set the output format to json before proceeding.
3. Go to the ECR service and open the public repositories section.
4. Start creating a new public repository by clicking on create repository.
5. Enter a suitable name for the public repository and complete the creation.
6. Open the created repository and check the push commands provided.
7. Use those commands to login, build the image, tag it, and push it to the public repository.
 - To build image --> `docker build -t <image_name> .`
 - Tag the image --> `docker tag <image_name> <public_repository_URI>:version`
 - Push the image --> `docker push <public_repository_URI>:version`
8. Confirm the image upload by running a container using the public image URI.

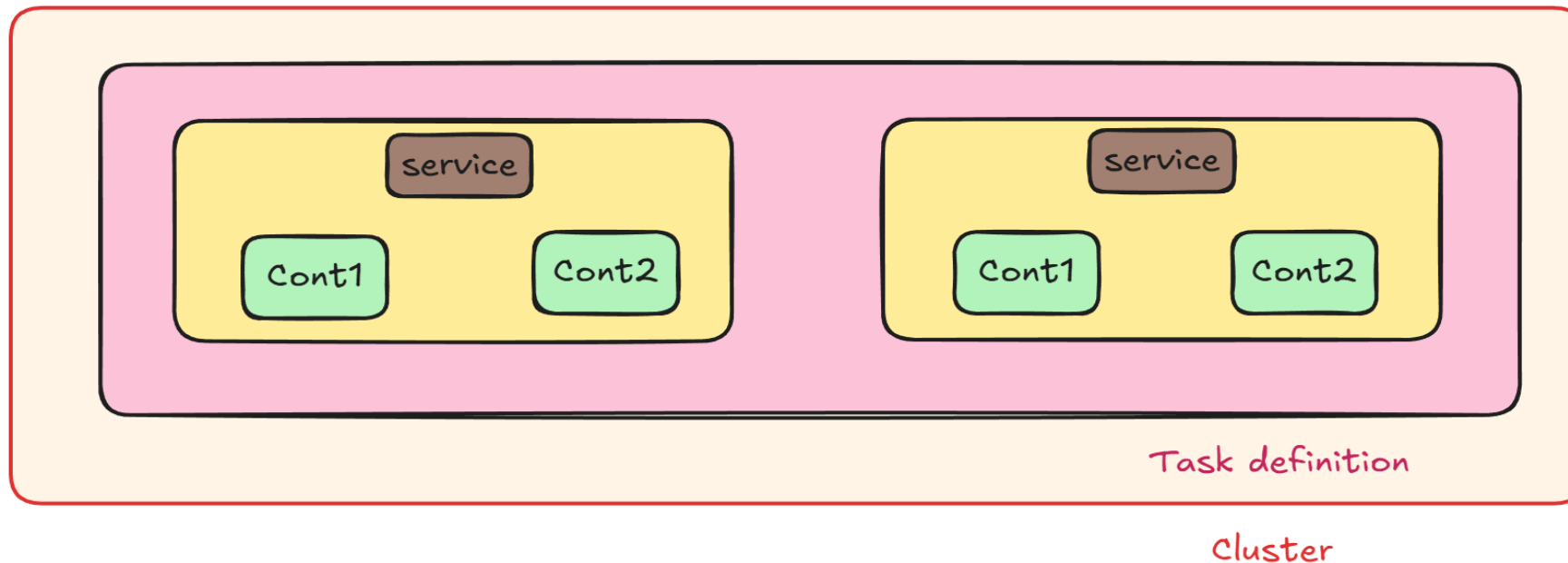
ECS – Elastic Container Service

- >> It is the service in AWS that is used to create containers and run the applications inside containers.
- >> ECS is the service that acts like a load manager which keeps the containers running in a stable state.

Why ECS is required?

1. To scale up or scale down whenever we need more memory or cpu load.
2. In case of high traffic, ecs automatically loads another container.
3. In order to distribute the load, you can connect to load balancer easily.

Architecture of ECS



Steps to implement ECS

1. Go to ECS in AWS and open the clusters section.
2. Click on create cluster and choose fargate as the infrastructure.
3. Enter a cluster name and complete the cluster creation.
4. Open task definitions and select create task definition.
5. Give a family name and move to container settings.
6. Provide the container name, add the image from ECR public gallery, ECR repo or dockerhub and set the port mapping if needed.
7. Create the task definition and ensure it is successfully created.
8. Deploy the task by creating a service, give the service name and finish the setup.
9. Verify that the service is created successfully.
10. Open the service and go to the running task.
11. Click the task ID and copy the public IP from the container configuration.
12. Open the browser and enter the public IP with the exposed port number.
 - URL --> `http://<public-ip>:<port-number>`